MLOPS Equipe 2

Projet d’option ASL-IA – Plateforme MLOps

Document d’architecture v0

Table des matières

Socle Physique et Cloud OpenStack 2
Cluster Kubernetes 3
Kubernetes, réseau, stockage 3
ML, données, MLflow 3
Serving 3
Observabilité et tests 3

Socle Physique et Cloud OpenStack

Hardware :

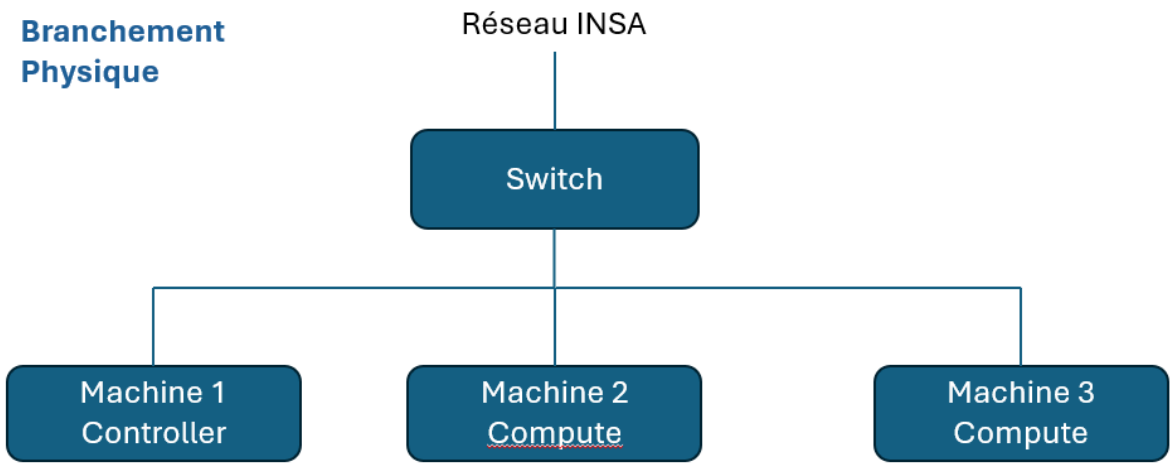
- Switch - HP A5120-48G-PoE+ EI ;
- Machines - 3 DELL OptiPlex 7040 ;

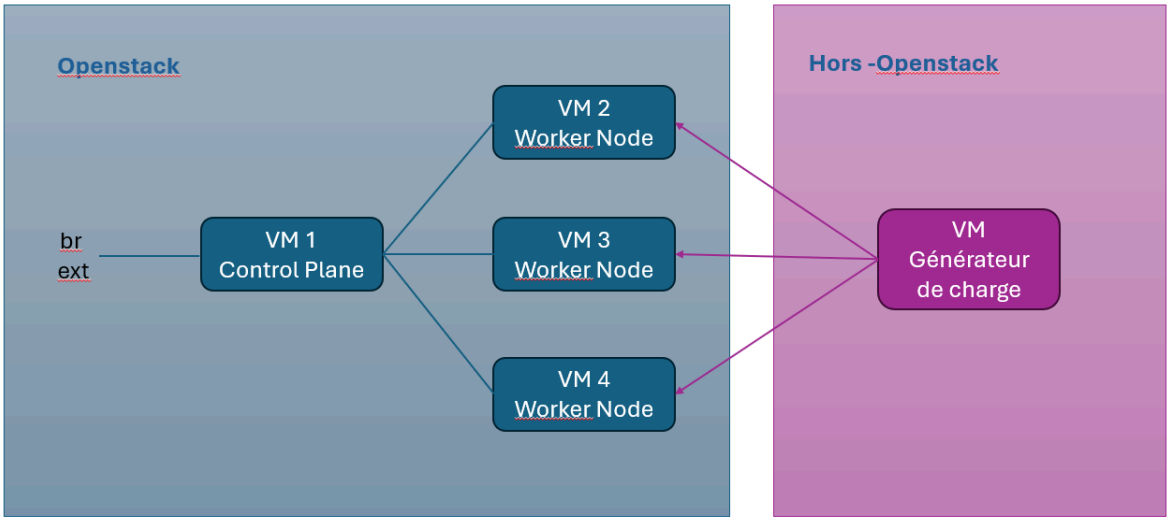
Software :

- Nous avons installé la version la plus récente de Debian (13.7) sur nos trois machines physiques ;
- Le DHCP Switch a correctement été configuré : L'internet est bien délivré à nos machines physiques.

Un profil Kolla-Ansible sera adopté, avec :

- 1 machine contrôleur OpenStack ;
- 2 machines computes.





Cluster Kubernetes

Kubernetes, réseau, stockage

Topologie :

Le cluster Kubernetes est déployé sur 4 VM OpenStack, plus une VM générateur de charge hors OpenStack :

- VM 1 : Control Plane (1 nœud) ;
- VM 2, 3, 4 : Worker Nodes (3 nœuds) ;
- VM générateur de charge (k6) : hors cluster et hors OpenStack, pour ne pas fausser les mesures.

Le dimensionnement indicatif est de 4 vCPU / 8 Go pour le Control Plane et de 4 vCPU / 12 Go par worker . Les VM sont créées via cloud-init depuis le tenant OpenStack dédié à l'équipe, avec accès SSH par floating IP ou bastion.

Installation :

- Distribution : kubeadm, automatisé avec Ansible (playbooks dans `infra/kubernetes/`). Plan B : k3s si kubeadm s'avère instable sur les VM.

- Versions de Kubernetes, du CNI et des composants figées dans `versions.yaml`, alignées sur la matrice de compatibilité KServe v0.14.0 / Knative.
- Namespaces : `mlops` (MLflow, PostgreSQL, stockage objet), `models` (InferenceServices), `monitoring` (Prometheus et Grafana).
- RBAC : ServiceAccounts et Roles dédiés par namespace.

Réseau :

Trois niveaux de réseau sont à distinguer :

- Réseau OpenStack (tenant) : les VM du cluster sont rattachées à un réseau privé dédié à l'équipe, avec un routeur vers le réseau externe (br-ext). Plan d'adressage : 10.10.0.0/24 ;
- Réseau des Pods : 10.244.0.0/16 ;
- Réseau des Services : 10.96.0.0/12.

CNI : Flannel . Flannel est le plus léger ; Calico n'est à privilégier que si des NetworkPolicies sont nécessaires.

Ingress : Kourier si Knative est installé (extension), sinon un NodePort ou ingress-nginx pour le RawDeployment. Le générateur de charge (hors OpenStack) atteint les workers via la floating IP et le port exposé.

Le réseau tenant relie les nœuds entre eux, le CNI gère le trafic entre pods par-dessus, et l'ingress est le seul point d'entrée pour les requêtes externes.

Points de vigilance :

- MTU : l'encapsulation Neutron (VXLAN/Geneve) et l'overlay du CNI réduisent la MTU utile. C'est une cause classique de blocages silencieux (TLS, pulls d'images). Elle est à vérifier dès la mise en place du cluster et à ajuster dans la configuration du CNI.
- Groupes de sécurité OpenStack : ils autorisent uniquement les flux nécessaires :
    - SSH (22) depuis le bastion ou la floating IP ;
    - API server (6443) depuis les nœuds et l'administrateur ;
    - kubelet (10250) entre les nœuds ;
    - etcd (2379-2380) entre les Control Planes uniquement ;
    - trafic du CNI entre les nœuds : UDP 8472 pour Flannel, UDP 4789 pour Calico en VXLAN ;
    - ports de l'ingress (NodePort 30000-32767 ou 80/443) depuis le générateur de charge.
- Tout le reste est refusé par défaut.

Stockage :

- StorageClass par défaut : `local-path-provisioner`, simple et suffisant pour le Niveau 1.
- Évolution possible (extension) : Cinder CSI pour attacher des volumes Cinder aux pods, avec snapshots.

- Validation (K8S-3) : un PVC est créé, des données sont écrites, le pod est redémarré, et les données doivent être conservées.
- Le stockage des artefacts de modèles passe par l'API S3 de RustFS (voir section ML), pas par les PVC.

Ressources et autoscaling :

- `requests` / `limits` définis sur chaque Deployment, et quotas par namespace.
- `metrics-server` installé pour `kubectl top` et pour le HPA.
- HPA sur CPU : observé en test sur le déploiement `housing-sklearn` (cible CPU 50 %, de 1 à 5 réplicas ; voir la section Observabilité et tests).

État d'avancement :

Les premiers tests ont été faits sur un cluster k3s mono-nœud créé sur un Proxmox personnel, en attendant la disponibilité d'OpenStack (démarrage indépendant des pôles). Le HPA et la montée en charge y ont été observés. Le cluster multi-nœuds sera construit sur les VM OpenStack avec les playbooks Ansible.

Choix :

- kubeadm + Ansible plutôt que k3s ou Kubespray : voir ADR-005, `docs/adr/ADR-005.md`.
- Flannel comme CNI : voir ADR-006, `docs/adr/ADR-006.md`.
- `local-path-provisioner` plutôt que Longhorn, NFS ou Cinder CSI : voir ADR-007, `docs/adr/ADR-007.md`.

ML, données, MLflow

Données :

Le jeu commun est California Housing (`sklearn.datasets.fetch_california_housing`) : 20 640 lignes, 8 variables dans l'ordre de l'annexe B (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude), cible `MedHouseVal` (valeur médiane des logements, en centaines de milliers de dollars).

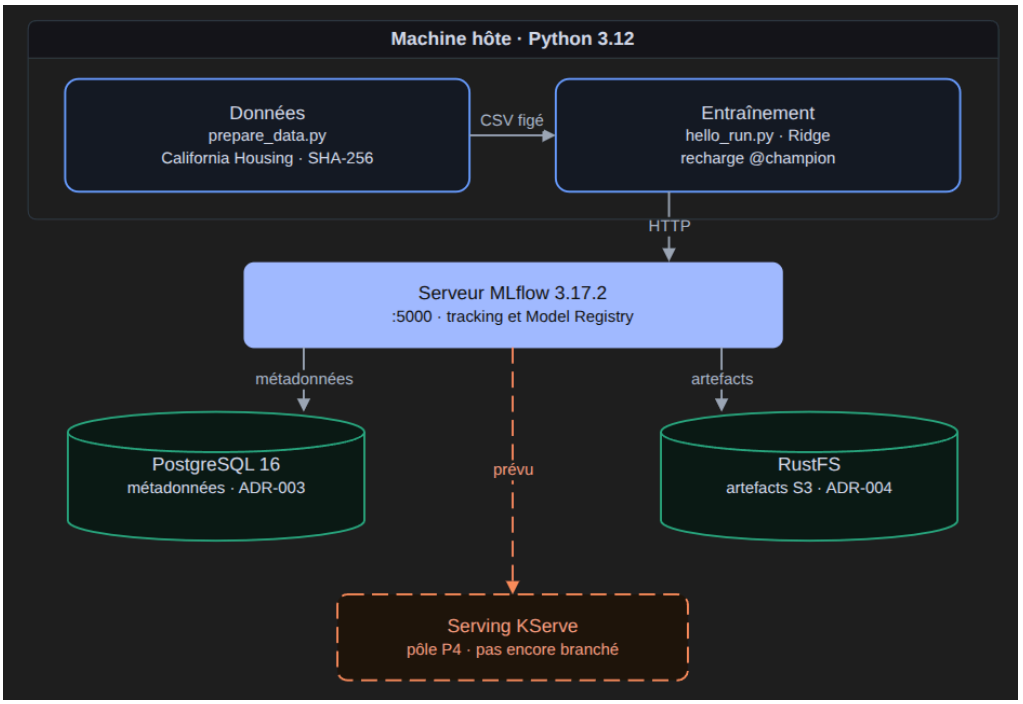
Le CSV est régénéré par `ml/src/prepare_data.py`. Seul le manifeste est versionné : empreinte SHA-256, nombre de lignes, liste des colonnes, versions de pandas 2.2.3 et de scikit-learn 1.5.1. Une empreinte différente bloque l'entraînement.

Stack locale (Docker Compose, `ml/compose.yaml`) :

- MLflow 3.17.2, interface `http://127.0.0.1:5000` ;
- PostgreSQL 16 : métadonnées des runs, des métriques et du Model Registry ;
- RustFS 1.0.0-alpha.83 : artefacts (modèles, fichiers) via une API S3, console `http://127.0.0.1:9001`, bucket `mlflow`.

Le client d'entraînement tourne sur la machine hôte en Python 3.12. Les versions sont figées dans `ml/requirements.txt` et alignées sur le runtime scikit-learn de KServe utilisé par le pôle Serving.

Workflow:



Choix :

- PostgreSQL plutôt que MySQL ou SQLite : voir ADR-003, `docs/adr/ADR-003.md`.
- RustFS plutôt que MinIO ou un disque local : voir ADR-004, `docs/adr/ADR-004.md`. MinIO communautaire n'est plus maintenu activement.
- Mêmes versions scikit-learn que KServe v0.14.0, pour que le modèle enregistré se recharge à l'identique au moment du serving.

Serving

Rôle et objectif : Le pôle Serving assure le déploiement opérationnel, l'exposition réseau et le cycle de vie des modèles d'inférence issus du Model Registry de MLflow. L'objectif pour le Niveau 1 (MVP) est de servir le modèle via une API d'inférence standardisée, interrogeable en JSON et renvoyant une prédiction cohérente avec les entraînements locaux.

Composants et architecture :

- Moteur de serving : KServe v0.14.0, déployé dans un premier temps en mode RawDeployment.
- Gestion des certificats : cert-manager v1.16.2, assurant la signature et l'injection des certificats TLS requis par les webhooks d'admission de KServe.
- Runtimes d'inférence : ClusterServingRuntimes standard de KServe, en particulier kserve-sklearnserver, aligné sur la version de scikit-learn utilisée par le pôle ML (1.5.1).
- Intégration stockage : Liaison avec le bucket S3 (RustFS / MinIO) géré par P3 via un Secret Kubernetes (identifiants S3) injecté dans un ServiceAccount dédié (sa-models). L'InferenceService utilise l'annotation storageUri pour que le conteneur d'initialisation de KServe télécharge automatiquement le modèle champion avant le démarrage du pod.

Mode de déploiement et stratégie d'évolution :

1. Niveau 1 (MVP - Décembre) : Mode RawDeployment. KServe instancie directement des ressources Kubernetes natives (Deployment, Service). Cette approche élimine toute dépendance critique vis-à-vis d'une couche de service mesh ou de Knative durant la phase de stabilisation initiale de l'infrastructure.
2. Niveau 2 (Extensions - Janvier) : Migration progressive vers le mode Serverless (Knative Serving + passerelle Kourier). Ce mode permettra d'évaluer le scale-to-zero, de mesurer le cold start (scénario B4) et de gérer la mise à l'échelle automatique basée sur la concurrence des requêtes (KPA).

Exposition et flux d'une requête :

1. Le client (générateur de charge k6 ou application cliente) émet une requête HTTP POST contenant la charge utile JSON vers l'Ingress ou le port exposé du cluster.
2. La requête est acheminée vers le Service Kubernetes (ou le VirtualService/Kourier en mode serverless) de l'InferenceService.
3. Le pod predictor reçoit la charge utile sur l'API protocole v1 (/v1/models/:predict), la transmet au modèle scikit-learn chargé en mémoire, calcule la prédiction et renvoie la réponse au format JSON.

État d’avancement :

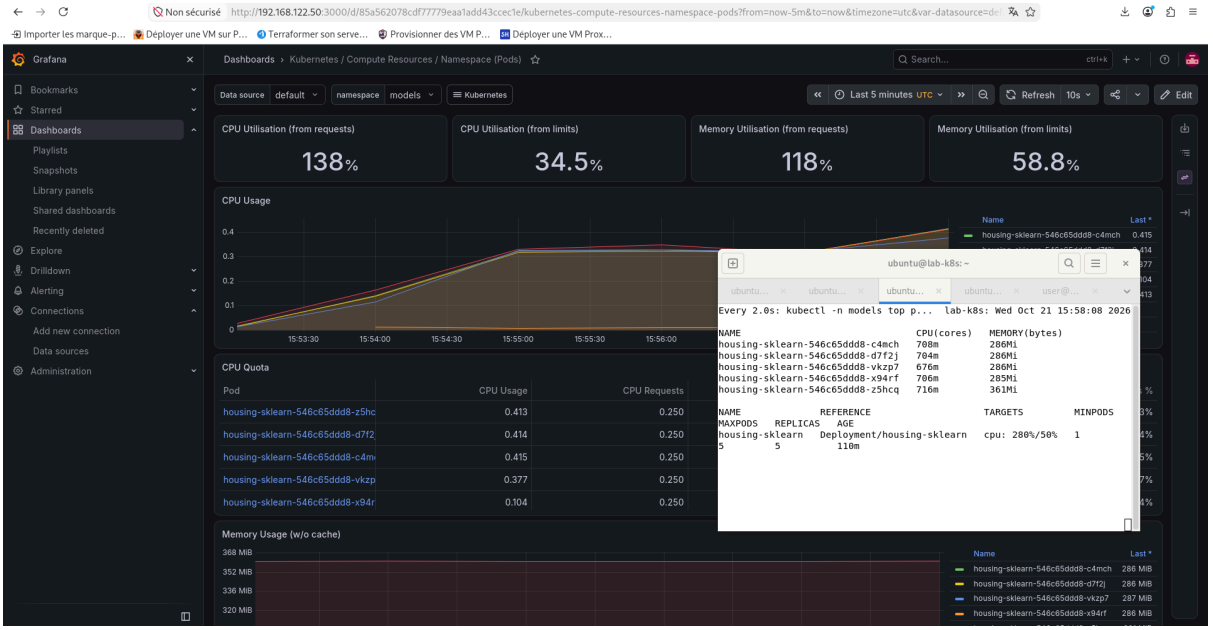
- Validation locale sur cluster kind (Debian) : Déploiement réussi de cert-manager v1.16.2 et de KServe v0.14.0 en mode RawDeployment.
- Contournement technique validé : Application des CRDs et manifests KServe via kubectl apply --server-side --force-conflicts afin de contourner le dépassement de taille limite des annotations de métadonnées Kubernetes sur la CRD inferenceservices.serving.kserve.io.
- Test fonctionnel de bout en bout : Déploiement d'un InferenceService témoin (sklearn-iris) et validation de l'appel d'inférence en protocole v1 renvoyant des prédictions conformes en local.
- Le dossier serving/ du dépôt Git contient le modèle de manifest cible, le README d'installation automatisée ainsi qu'un échantillon de payload JSON prêt pour les tests d'intégration avec P3.

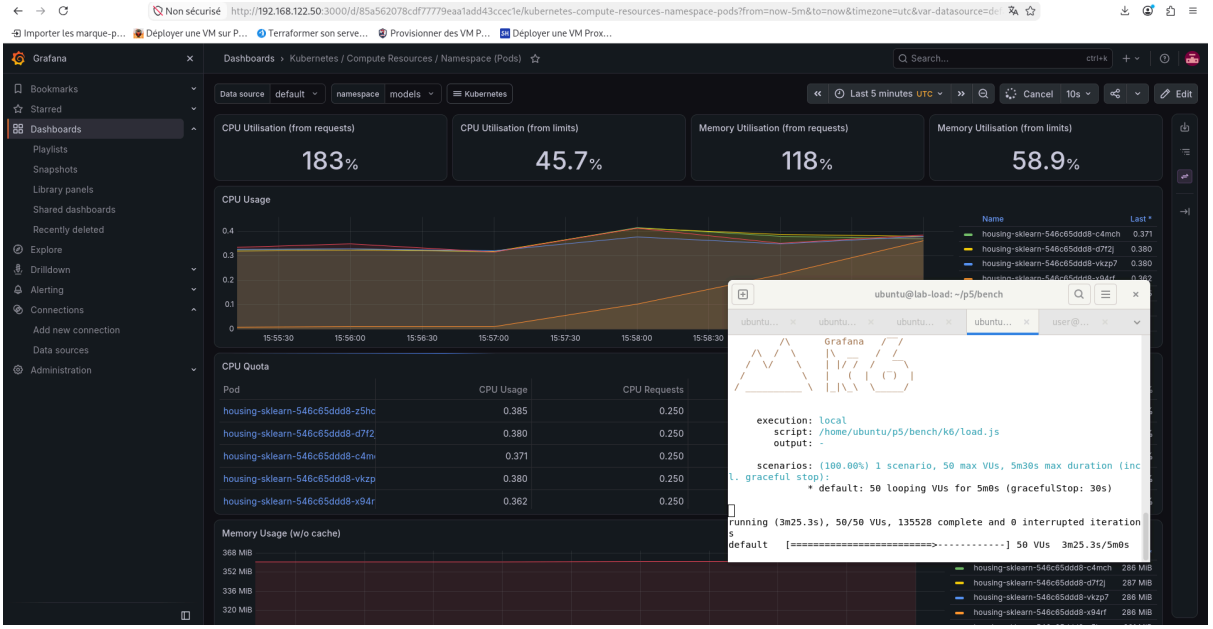
Choix et ADR :

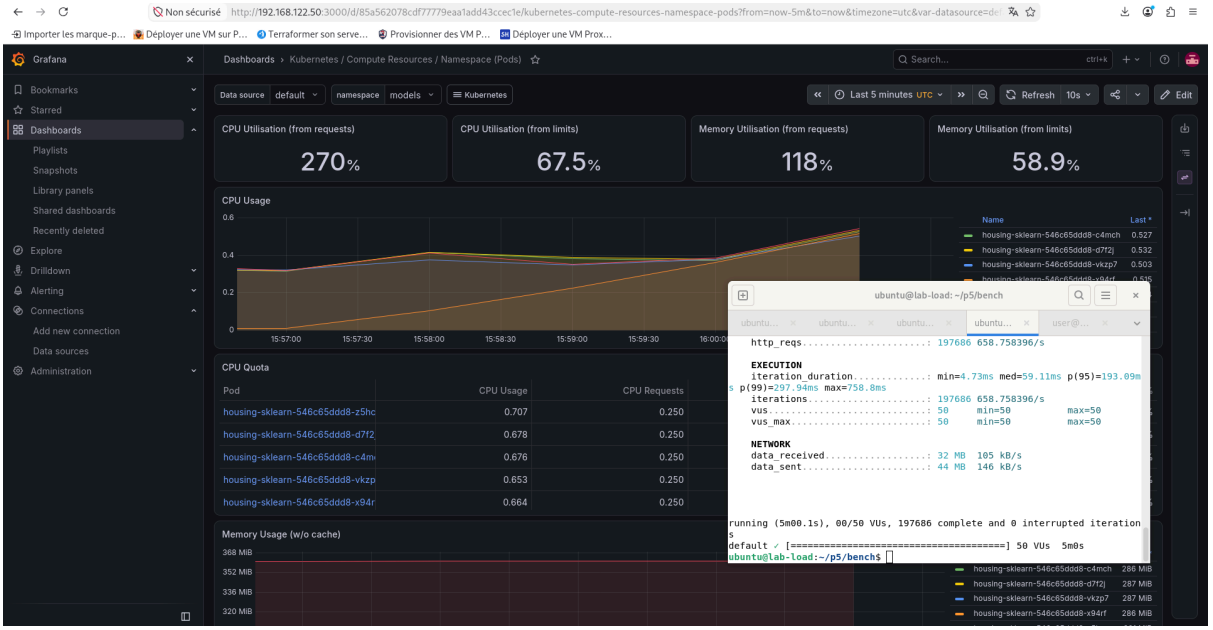
- KServe en mode RawDeployment pour le MVP plutôt que FastAPI seul ou Knative complet d'emblée : voir ADR-008 (docs/adr/ADR-008.md). Le mode RawDeployment garantit l'atteinte du jalon J5 en évitant les surcoûts et risques réseau liés à Knative/Istio, tout en conservant le standard d'API de KServe.
- Protocole de serving KServe v1 (KFServing Dataplane v1) pour la compatibilité native avec les runtimes scikit-learn.

Observabilité et tests

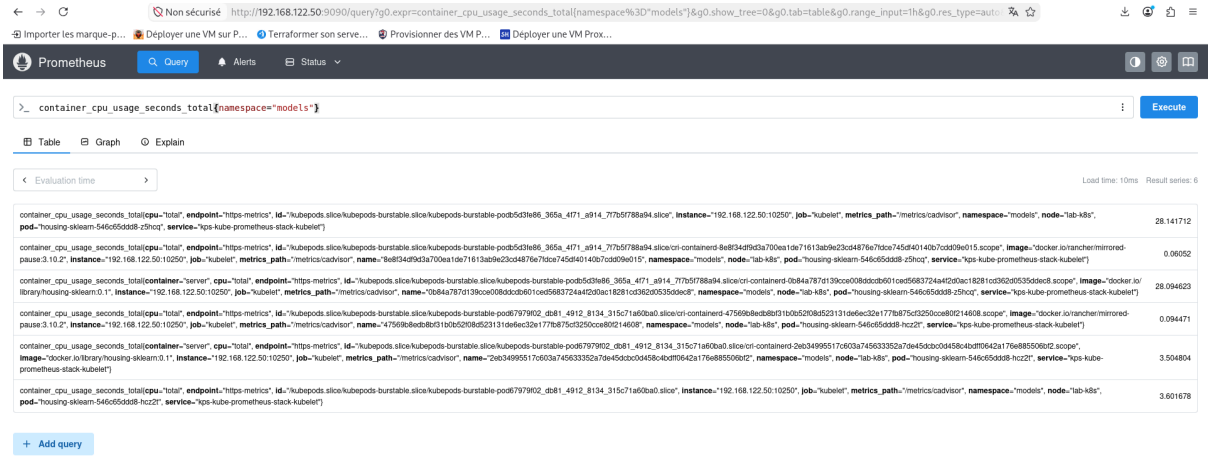
Test fait en local via des machines Kubernetes créées sur le Proxmox personnel.
Résultat de l’installation de Grafana + Prometheus et test de charges :


Non sécurisé http://192.168.122.50:3000/d/85a562078cdf77779eaa1add43ccec1e/kubernetes-compute-resources-namespace-pods?from=now-5m&to=now&timezone=utc&var-datasource=de
Importer les marque-p… Déployer une VM sur P… Terraformer son serve… Provisionner des VM P… Déployer une VM Prox…
Grafana
Dashboards › Kubernetes / Compute Resources / Namespace (Pods)
Search… ctrl+k
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Data source default
namespace models
Kubernetes
Last 5 minutes UTC
Refresh 10s
Edit
CPU Utilisation (from requests)
138%
CPU Utilisation (from limits)
34.5%
Memory Utilisation (from requests)
118%
Memory Utilisation (from limits)
58.8%
CPU Usage
Name
Last *
housing-sklearn-546c65ddd8-c4mch 0.415
15:53:30
15:54:00
15:54:30
15:55:00
15:55:30
15:56:00
ubuntu@lab-k8s: ~
Every 2.0s: kubectl -n models top p... lab-k8s: Wed Oct 21 15:58:08 2026
NAME CPU(cores) MEMORY(bytes)
housing-sklearn-546c65ddd8-c4mch 708m 286Mi
housing-sklearn-546c65ddd8-d7f2j 704m 286Mi
housing-sklearn-546c65ddd8-vkzp7 676m 286Mi
housing-sklearn-546c65ddd8-x94rf 706m 285Mi
housing-sklearn-546c65ddd8-z5hcq 716m 361Mi
NAME REFERENCE TARGETS MINPODS
MAXPODS REPLICAS AGE
housing-sklearn Deployment/housing-sklearn cpu: 280%/50% 1
5 5 110m
CPU Quota
Pod
CPU Usage
CPU Requests
housing-sklearn-546c65ddd8-z5hc 0.413 0.250
housing-sklearn-546c65ddd8-d7f2 0.414 0.250
housing-sklearn-546c65ddd8-c4m 0.415 0.250
housing-sklearn-546c65ddd8-vkzp 0.377 0.250
housing-sklearn-546c65ddd8-x94r 0.104 0.250
Memory Usage (w/o cache)
368 MiB
352 MiB
336 MiB
320 MiB
housing-sklearn-546c65ddd8-c4mch 286 MiB
housing-sklearn-546c65ddd8-d7f2j 286 MiB
housing-sklearn-546c65ddd8-vkzp7 287 MiB
housing-sklearn-546c65ddd8-x94rf 286 MiB


Non sécurisé http://192.168.122.50:3000/d/85a562078cdf77779eaa1add43ccec1e/kubernetes-compute-resources-namespace-pods?from=now-5m&to=now&timezone=utc&var-datasource=de
Importer les marque-p… Déployer une VM sur P… Terraformer son serve… Provisionner des VM P… Déployer une VM Prox…
Grafana
Dashboards › Kubernetes / Compute Resources / Namespace (Pods)
Search… ctrl+k
Data source default
namespace models
Kubernetes
Last 5 minutes UTC
Cancel 10s
Edit
CPU Utilisation (from requests)
183%
CPU Utilisation (from limits)
45.7%
Memory Utilisation (from requests)
118%
Memory Utilisation (from limits)
58.9%
CPU Usage
Name
Last *
housing-sklearn-546c65ddd8-c4mch 0.371
housing-sklearn-546c65ddd8-d7f2j 0.380
housing-sklearn-546c65ddd8-vkzp7 0.380
15:55:30
15:56:00
15:56:30
15:57:00
15:57:30
15:58:00
15:58:30
ubuntu@lab-load: ~/p5/bench
Grafana
execution: local
script: /home/ubuntu/p5/bench/k6/load.js
output: -
scenarios: (100.00%) 1 scenario, 50 max VUs, 5m30s max duration (incl. graceful stop):
* default: 50 looping VUs for 5m0s (gracefulStop: 30s)
running (3m25.3s), 50/50 VUs, 135528 complete and 0 interrupted iterations
default [==========>----------] 50 VUs 3m25.3s/5m0s
CPU Quota
Pod
CPU Usage
CPU Requests
housing-sklearn-546c65ddd8-z5hc 0.385 0.250
housing-sklearn-546c65ddd8-d7f2 0.380 0.250
housing-sklearn-546c65ddd8-c4m 0.371 0.250
housing-sklearn-546c65ddd8-vkzp 0.380 0.250
housing-sklearn-546c65ddd8-x94r 0.362 0.250
Memory Usage (w/o cache)
368 MiB
352 MiB
336 MiB
320 MiB
housing-sklearn-546c65ddd8-c4mch 286 MiB
housing-sklearn-546c65ddd8-d7f2j 287 MiB
housing-sklearn-546c65ddd8-vkzp7 286 MiB
housing-sklearn-546c65ddd8-x94rf 286 MiB



On se retrouve avec un environnement observable depuis l'interface web Grafana.
Une machine extérieure envoie des requêtes aux différents pods k8s, prométhéum récupère les informations et Grafana s'occupe de la mise en forme via graphique.
Prometheus :



Choix:
Pourquoi Grafana et prometheus et non les solutions alternatives : voir adr-002: [projet-mlops-equipe-2/docs/adr/ADR-002-Test_et_Observabilité.md at main · maxgigi/projet-mlops-equipe-2 · GitHub]